# Software Requirements and Design Document

# For

# Group 14

Version 2.0

**Authors**:
Samantha Bui
Ludginie Dorval
Antonio Garriga
William Lee
Lillian Malik

## 1. Overview (5 points)

*ManuFactor* is a web-based software program designed to simplify key accounting tasks for manufacturing businesses. The platform automates processes such as Cost-Volume-Profit (CVP) analysis and production budgeting (BP), making it easier for companies to plan production, set sales targets, and achieve income goals. The system features three distinct user roles to control access and functionality: Admins, who can manage both products and user data; Users, who can manage products but have restricted access to user data; and Guests, who can only view product information. This role-based structure ensures the appropriate level of access for different types of users while maintaining a secure and efficient system.

## 2. Functional Requirements (10 points)

1. **User Management (High):** refers to how different roles in the system (Admin, User, Viewer) manage users within the application. The Admin role is responsible for overseeing all aspects of user management, which includes creating, reading, updating, and deleting user accounts. The User role is limited to creating, reading, updating, and deleting specific data, while the Viewer role is limited to reading data only without modification capabilities.
   - Admin Role:
     - Ability to create, update, and delete user accounts.
   - User Role:
     - Can view their assigned product data but cannot modify user-related data or access other users' information.
   - Guest Role:
     - Can view publicly available product data, but has no ability to interact with or modify any data.
2. **Product Management (High):** refers to how different roles manage product data within the system. The Admin role holds full control over product management and is responsible for creating, reading, updating, and deleting product data, while the User role is responsible for creating, reading, updating, and deleting product data. Viewers have a read-only role for product data.
   - Admin Role:
     - Ability to add, update, and delete product data.
   - User Role:
     - Ability to add, update, and delete product information.
     - Users can view product data that is assigned to them but cannot access products that belong to other companies.
   - Guest Role:
     - Can only view a list of products with basic details but cannot modify or add new products.
3. **Cost-Volume-Profit (CVP) Analysis (**High**)**
   - Admins and Users should be able to input data such as fixed cost, variable cost per unit, selling price per unit, and target income for each product.
   - Ability to generate and view CVP analysis reports based on user-provided data.
4. **Production Budgeting (**High**)**
   - Admins and Users should be able to input data such as desired ending inventory, direct material per unit, direct material price, desired ending inventory, and the projected sales for the current, next, and subsequent months for each product.
   - Ability to generate and view BP analysis reports based on user-provided data.
5. **Dashboard (**Medium**)**
   - Provide a dashboard that displays depending on the user’s security level, such as:
     - For Admin: Overview of all users, products, and financial data.
     - For User: Overview of the products and related financial data.
     - For Viewers: View available products and basic product details.
6. **Access Control and Permissions (**Medium**)**
   - Implement role-based access control to ensure that users only have access to data and features appropriate for their role.
     - Admins can view and modify all data, while Users are restricted to managing only their assigned products.

    - Guests have read-only access to product information and cannot make changes to any data.

7. **Security and Authentication (**Medium**)**
   - Implement secure login functionality using username and password.
     - Potentially including email as well.
8. **Data Storage and Backup (**Medium**)**
   - The system should store all user, product, financial data, and reports securely in a database.
9. **User Interface and Experience (**Low**)**
   - The application should have a clean, intuitive, and responsive user interface.
   - The interface should adapt for different devices (desktop, tablet, mobile) for easy access and use.
   - Provide easy navigation between user profiles, product management, financial reports, and system settings.

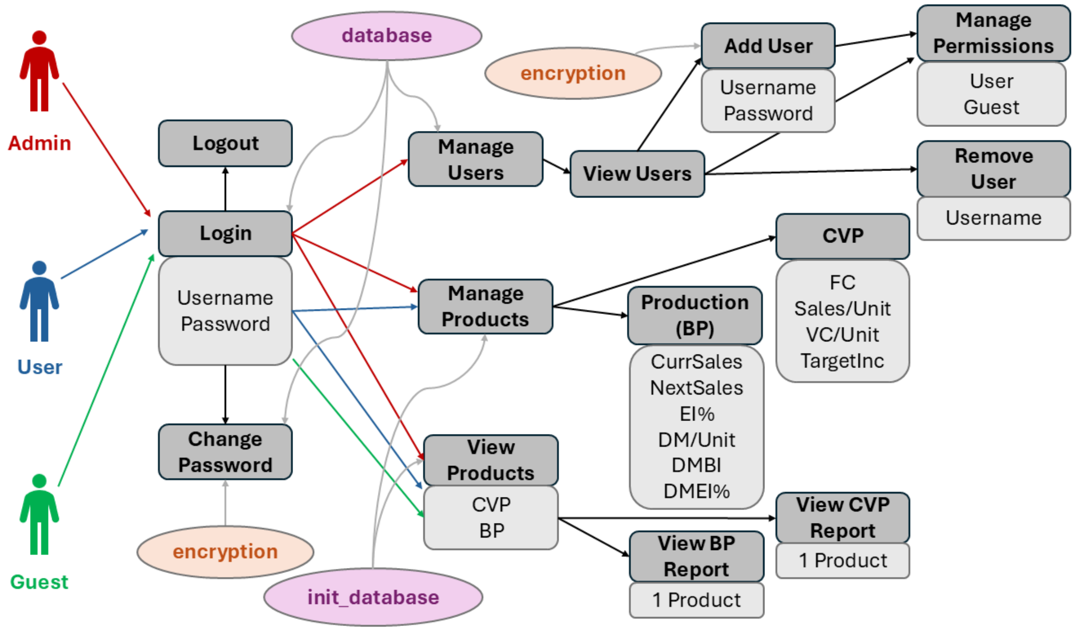
## 3. Non-functional Requirements (10 points)

1. **Performance**
   - Response Time: The system should load within 3 seconds for most operations (e.g., navigating between pages, generating reports).
   - Throughput: The system should be able to handle up to 20 concurrent users without performance degradation.
2. **Scalability**
   - The system should be scalable to accommodate future growth in the number of users and products without major changes to its structure.
   - It should be able to support an increasing number of products, users, and financial data.
3. **Security**
   - Data Encryption: All sensitive data should be encrypted (password).
   - Authentication: Implement role-based access control to ensure only authorized users have access to specific features and data.
   - Authorization: Ensure that users can only access and modify data according to their assigned roles (Admins, Users, and Guests).
   - Data Privacy: The system must ensure user data is protected.
4. **Usability**
   - User Interface (UI): The system must be easy to navigate, with a clean and intuitive design, so users can quickly find and use features.
   - Responsive Design: The application should be fully responsive, ensuring it works on various devices, including desktops, tablets, and smartphones.
5. **Maintainability**
   - Code Quality: The codebase should follow best practices for clean, modular, and well-documented code to allow for easy updates for future increments.
   - Testing: Comprehensive tests should be implemented to ensure reliability and quality.
6. **Modularity**
   - Extensibility: The system should be designed with extensibility in mind, so new features can be easily added.
7. **Compatibility**
   - Cross-Browser Compatibility: The system should be fully compatible with the latest versions of major web browsers.
   - Operating System Compatibility: The system should be compatible with commonly used operating systems.
8. **Error Handling:**

   The system should provide meaningful feedback when an error occurs (e.g invalid login)
9. **Portability:**

   The system should be easily deployable across different machines with minimal setup. All necessary database initialization is handled programmatically, making the application ready to run without manual SQL imports.

## 4. Use Case Diagram (10 points)




Admin Role:

- Ability to create, update, and delete user accounts.

User Role:

- Can view their assigned product data but cannot modify user-related data or access other users' information.

Guest Role:

- Can view publicly available product data, but cannot interact with or modify any data.

Each user should be able to log in, log out, and change their password (to be implemented in Increment 3 due to priority level). Each time they change their password or a new user is added, their private information is encrypted and kept secure. The front-end and back-end databases are called for when their respective data is called into use, for example, database.py is for the front-end information (user management), and init_database.sql for the back-end information (product management). In this increment, we added the BP and CVP reports, which offer a detailed summary of the data for one product.

…

## 5. Class Diagram and/or Sequence Diagrams (15 points)

<u>**FRONT-END CLASS DIAGRAM (*app.py*)**</u>

**Flask App**

- app: Flask
- secret_key: str

+ login(): Response
+ logout(): Response
+ home(): Response
+ cvp_results(): Response
+ bp_results(): Response
+ list_data(): Response
+ add_data(): Response
+ list_users(): Response

↓
**Has**
↓

**User**

- ID: int
- username: str
- password: str
- security: int

+ authenticate(): bool
+ encrypt_data(): str
+ decrypt_data(): str

↓
**Interacts With**
↓

**Database**

+ initialize_database(): None
+ add_user_to_database(name: str, security_level: int, password: str): None
+ get_username_from_database(encrypted_username: str): dict

↓
**Uses**
↓

**Cipher**

+ encrypt(data: bytes): str
+ decrypt(data: str): str

↓
**Uses**
↓

**Session**

- ID: int
- username: str
- security: int

+ set_session_data(): None
+ clear_session(): None

**FRONT-END DATABSE CLASS DIAGRAM (*database.py*)**

| **Database** |
|---|
| - connection: mysql.connector<br>- cursor: mysql.connector.cursor |
| + initialize_database(): void<br>+ add_user_to_database(username: str, password: str, security_level: int): void<br>+ get_username_from_database(encrypted_username: str): dict |

↓
**Uses**
↓

| **Cipher** |
|---|
| + encrypt(data: bytes): str<br>+ decrypt(data: str): str |

**ENCRYPTION CLASS DIAGRAM (*encryption.py*)**

| **AESCipher** |
|---|
| - key: bytes<br>- iv: bytes |
| + init(key: bytes, iv: bytes): void<br>+ encrypt(data: bytes): str<br>+ decrypt(data: str): str |

**BACK-END CLASS DIAGRAM (*product_cvp.py*)**

| **DatabaseConnection** |
|---|
| - host: str<br>- user: str<br>- password: str<br>- database: str<br>- connection: mysql.connector.connect()<br>- cursor: mysql.connector.cursor() |
| + connect()<br>+ execute(query: str, values: tuple)<br>+ commit()<br>+ close()<br>+ get_product_cvp(company_id: int): dict<br>+ update_product_cvp(…) |

↓
**Uses**
↓

| **BreakevenCalculator** |
|---|
| - fixed_cost: float<br>- variable_cost_per_unit: float<br>- selling_price_per_unit: float<br>- target_income: float |
| + calculate_breakeven_and_target_sale(data: dict): dict |

↓
**Calls**
↓

| **MainProgram** |
| --- |
| - company_id: dict<br>- product_data: dict<br>- breakeven_units: float<br>- breakeven_dollars: float<br>- target_units: float<br>- target_dollars: float |
| + get_product_cvp(company_id: int)<br>+ update_product_cvp(…)<br>+ calculate_breakeven_and_target_sales(data: dict)<br>+ print_results(result: dict)<br>+ main() |

## BACK-END CLASS DIAGRAM (*product_bp.py*)

| **DatabaseConnection** |
| --- |
| - host: str<br>- user: str<br>- password: str<br>- database: str<br>- connection: mysql.connector.connect()<br>- cursor: mysql.connector.cursor() |
| + connect()<br>+ execute(query: str, values: tuple)<br>+ commit()<br>+ close()<br>+ get_product_cvp(company_id: int): dict<br>+ update_product_cvp(…) |

↓
**Uses**
↓

| **ProductionCalculator** |
| --- |
| - ei_rate: float<br>- curr_sales: float<br>- next_sales: float<br>- twicenext_sales: float<br>- dm_per_unit: float<br>- ei_dm_rate: float<br>- dm_price: float |
| + calc_production(curr_sales, next_sales, ei_rate)<br>+ calc_purchases(curr_production, next_production, ei_dm_rate, dm_per_unit, dm_price) |

↓
**Calls**
↓

| **MainProgram** |
| --- |
| - company_id: dict<br>- data: dict<br>- curr_sales: float<br>- next_sales: float<br>- twicenext_sales: float<br>- EI_Rate: float<br>- DM_per_Unit: float<br>- EI_DM_Rate: float<br>- DM_Price: float |

| |
|---|
| - this_month_production: float<br>- next_month_production: float<br>- dm_purchases: float |
| + get_product_bp(company_id: int)<br>+ update_product_bp(…)<br>+ calc_production(…)<br>+ calc_purchases(…)<br>+ main() |

**<u>BACK-END DATABSE CLASS DIAGRAM (*init_database.sql*)</u>**

| **user** |
|---|
| - user_id: int<br>- comp_id: int<br>- fname: VARCHAR(15)<br>- lname: VARCHAR(15)<br>- email: VARCHAR(20)<br>- username: VARCHAR(225) (UNIQUE)<br>- security_level: int<br>- password: VARCHAR(225) |

| **product_cvp** |
|---|
| - prod_id: int<br>- comp_id: int<br>- fixed_cost: DECIMAL(15,2)<br>- variable_cost_per_unit: DECIMAL(10,2)<br>- selling_price_per_unit: DECIMAL(10,2)<br>- target_income: DECIMAL(15,2) |

| **product_bp** |
|---|
| - prod_id: int<br>- comp_id: int<br>- curr_sales: int<br>- next_sales: int<br>- twicenext_sales: int<br>- EI_rate: DECIMAL(10,2)<br>- DM_per_unit: int<br>- EI_DM_rate: DECIMAL(10,2)<br>- DM_price: DECIMAL(10,2) |

**<u>ADMIN SEQUENCE DIAGRAM:</u>**

Adding Users/Managing Products

| **Login** |
|---|
| Admin -> Flask App: login()<br>Flask App -> User: authenticate(username, password)<br>User -> Database: get_username_from_database(encrypted_username)<br>Database -> Cipher: decrypt(data)<br>Cipher -> Database: return decrypted data<br>Database -> User: return authentication result<br>User -> Flask App: return authentication result<br>Flask App -> Session: set_session_data()<br>Flask App -> Admin: redirect to home() |

**…**

| **View User Data** |
| --- |
| Admin -> Flask App: list_users() |
| Flask App -> Database: get_users() |
| Database -> Cipher: decrypt(user_data) |
| Cipher -> Database: return decrypted data |
| Database -> Flask App: return user data |
| Flask App -> Admin: return user data (list) |

| **Add New User** |
| --- |
| Admin -> Flask App: add_data(name, security_level, password) |
| Flask App -> Database: add_user_to_database(name, security_level, encrypted_password) |
| Database -> Cipher: encrypt(password) |
| Cipher -> Database: return encrypted password |
| Database -> Flask App: return success |
| Flask App -> Admin: confirm user added |

| **Manage Products** |
| --- |
| Admin -> Flask App: list_data() |
| Flask App -> Database: get_product_cvp(company_id) |
| Database -> Flask App: return product data |
| Flask App -> BreakevenCalculator: calculate_breakeven_and_target_sale(data) |
| BreakevenCalculator -> Flask App: return CVP analysis results |
| Flask App -> Admin: display CVP analysis results |

**USER SEQUENCE DIAGRAM:**

Viewing Data/CVP Analysis

| **Login** |
| --- |
| User -> Flask App: login() |
| Flask App -> User: authenticate(username, password) |
| User -> Database: get_username_from_database(encrypted_username) |
| Database -> Cipher: decrypt(data) |
| Cipher -> Database: return decrypted data |
| Database -> User: return authentication result |
| User -> Flask App: return authentication result |
| Flask App -> Session: set_session_data() |
| Flask App -> User: redirect to home() |

| **View Product Data** |
| --- |
| User -> Flask App: list_data() |
| Flask App -> Database: get_product_cvp(company_id) |
| Database -> Flask App: return product data |
| Flask App -> User: return product data (list) |

| **Perform CVP Analysis** |
| --- |
| User -> Flask App: results() |
| Flask App -> Database: get_product_cvp(company_id) |
| Database -> Flask App: return product data |
| Flask App -> BreakevenCalculator: calculate_breakeven_and_target_sale(data) |
| BreakevenCalculator -> Flask App: return CVP analysis results |
| Flask App -> User: display CVP analysis results |

…

**<u>GUEST SEQUENCE DIAGRAM:</u>**

Viewing Only

| **Login** |
|---|
| Guest -> Flask App: login()<br>Flask App -> User: authenticate(username, password)<br>User -> Database: get_username_from_database(encrypted_username)<br>Database -> Cipher: decrypt(data)<br>Cipher -> Database: return decrypted data<br>Database -> User: return authentication result<br>User -> Flask App: return authentication result<br>Flask App -> Session: set_session_data()<br>Flask App -> Guest: redirect to home() |

| **View Product Data** |
|---|
| Guest -> Flask App: list_data()<br>Flask App -> Database: get_product_cvp(company_id)<br>Database -> Flask App: return product data<br>Flask App -> Guest: return product data (view only) |

| **View CVP Analysis Results** |
|---|
| Guest -> Flask App: results()<br>Flask App -> Database: get_product_cvp(company_id)<br>Database -> Flask App: return product data<br>Flask App -> BreakevenCalculator: calculate_breakeven_and_target_sale(data)<br>BreakevenCalculator -> Flask App: return CVP analysis results<br>Flask App -> Guest: display CVP analysis results |

## 6. Operating Environment (5 points)

1. **<u>Hardware Platform:</u>**
   - Users can access the system on desktops, laptops, tablets, and mobile devices, provided they have an internet connection.
2. **<u>Operating System:</u>**
   - Since the software will be accessed through a web browser, it is compatible with most major operating systems.
3. **<u>Web Browser Compatibility:</u>**
   - The web application will support the latest versions of commonly used browsers, and older versions may not be compatible or cause issues for the user.
4. **<u>Software Components:</u>**
   - <u>Frontend</u>:
     - The application will be built using HTML, CSS, and JavaScript.
     - The front end communicates with the backend using JSON for data exchange.
   - <u>Backend</u>:
     - The backend of *ManuFactor* is a Python-based server (using Flask) for efficient data handling and user requests.
     - The system relies on an SQL-based database (using MySQL) to store user data, product information, and financial records.

…

## 7. Assumptions and Dependencies (5 points)

1. **Assumptions:**
    - User Access to Modern Browsers: It is assumed that users of *ManuFactor* will have access to modern web browsers and will keep them up to date.
    - Stable Internet Connectivity: The application is a web-based system, so it assumes stable internet access for users to interact with the software.
    - Sufficient Hardware for End Users: It is assumed that end users will be using devices that can run modern web browsers.
    - If any of these assumptions are incorrect, the web page cannot be reached.
2. **Dependencies**:
    - Database Management Systems: The project assumes that the SQL-based database (MySQL) will be stable and perform well. Any issues in the database management system could impact performance.
    - Security Tools: The project will depend on services for encryption (Cipher). Any issues regarding Cipher could pose security risks.
3. **Constraints**:
    - Development Timeline: The project timeline is relatively short, which could affect the project's quality.